

T1 Fundamentos

Índice

1. Introducción
2. Análisis de componentes principales (PCA)
3. Gradiente
4. Jacobiana
5. Descenso por gradiente
6. Cálculo matricial
7. Optimización con restricciones
8. Distribución de Bernoulli
9. Distribución categórica
10. Distribución Gaussiana
11. Entropía
12. Estimación máximo-verosímil (MLE)
13. Minimización del riesgo empírico (ERM)
14. Regularización
15. Decisión

1 Introducción

Aprendizaje automático (ML): un sistema aprende de la experiencia E respecto a una clase de tareas T y una medida de rendimiento R , si su rendimiento en T , medido por R , mejora con E

Aproximación probabilística: el sistema se basa en un **modelo probabilístico** aprendido a partir de **datos**

Tareas de clasificación: modelo $p(y \mid \mathbf{x}; \boldsymbol{\theta})$, entrada $\mathbf{x} \in \mathbb{R}^D$, salida $y \in \{1, \dots, C\}$ y parámetros $\boldsymbol{\theta}$

Tareas de regresión: modelo $p(y \mid \mathbf{x}; \boldsymbol{\theta})$, entrada $\mathbf{x} \in \mathbb{R}^D$, salida $y \in \mathbb{R}$ y parámetros $\boldsymbol{\theta}$

Aprendizaje supervisado: disponemos de un conjunto de datos supervisados o etiquetados, $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$

Fundamentos: algebra lineal, optimización, probabilidad, información, estadística y decisión

Objetivo: introducir fundamentos esenciales en ML

2 Análisis de componentes principales (PCA)

Maldición de la dimensionalidad: muchas técnicas de ML empeoran sensiblemente con datos de alta dimensión

Reducción de la dimensión: dada una matriz de N datos en $\mathbb{R}^D$, queremos aprender (no supervisadamente) una transformación de $\mathbb{R}^D$ en $\mathbb{R}^K$, $K \ll D$, que "aproxime bien" los datos originales

Codificación: operación de reducción de la dimensión, $\text{encode}(\mathbf{x}) = \mathbf{z}$, $\mathbf{x} \in \mathbb{R}^D$ y $\mathbf{z} \in \mathbb{R}^K$

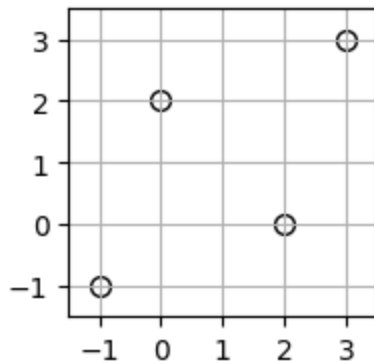
Decodificación: operación de reconstrucción del dato original, $\text{decode}(\mathbf{z}) = \hat{\mathbf{x}}$, $\hat{\mathbf{x}} \in \mathbb{R}^D$ y $\mathbf{z} \in \mathbb{R}^K$

PCA: escoge una proyección lineal ortogonal $\mathbf{W} \in \mathbb{R}^{D \times K}$ de mínima **distorsión o error de reconstrucción**

$$\mathcal{L}(\mathbf{W}) = \frac{1}{N} \sum_n \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n))\|_2^2 \quad \text{con} \quad \text{encode}(\mathbf{x}) = \mathbf{W}^t \mathbf{x} \quad \text{y} \quad \text{decode}(\mathbf{z}) = \mathbf{W} \mathbf{z}$$

Ejercicio: codifica, decodifica y halla la distorsión de los siguientes datos con $\mathbf{W} = (1, 0)^t$ y $\mathbf{W} = (\sqrt{2}/2, \sqrt{2}/2)^t$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); N = len(X)
fig, ax = plt.subplots(figsize=(2, 2)); ax.set_aspect("equal"); plt.axis([-1.5, 3.5, -1.5, 3.5])
plt.grid(True); plt.xticks(range(-1,4)); plt.scatter(*X.T, facecolor='white', edgecolor='k', s=50);
```

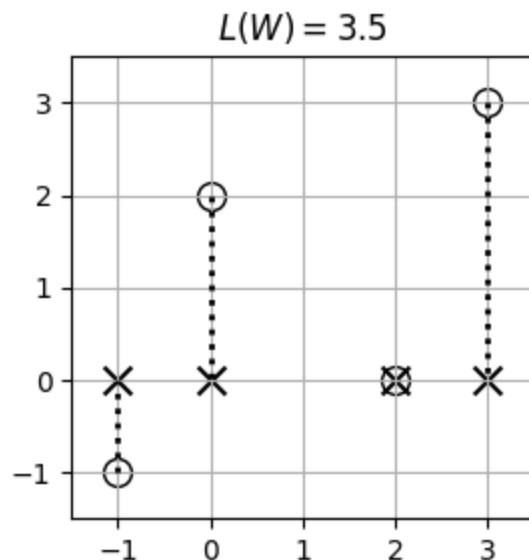


$$\mathbf{W} = (1, 0)^t:$$

$$\begin{array}{llll} \mathbf{x}_1 = (-1, -1)^t & \mathbf{x}_2 = (0, 2)^t & \mathbf{x}_3 = (2, 0)^t & \mathbf{x}_4 = (3, 3)^t \\ z_1 = \mathbf{W}^t \mathbf{x}_1 = -1 & z_2 = 0 & z_3 = 2 & z_4 = 3 \\ \hat{\mathbf{x}}_1 = \mathbf{W} z_1 = (-1, 0)^t & \hat{\mathbf{x}}_2 = (0, 0)^t & \hat{\mathbf{x}}_3 = (2, 0)^t & \hat{\mathbf{x}}_4 = (3, 0)^t \end{array}$$

$$\mathcal{L}(\mathbf{W}) = \frac{1}{4} (\|\mathbf{x}_1 - \hat{\mathbf{x}}_1\|_2^2 + \|\mathbf{x}_2 - \hat{\mathbf{x}}_2\|_2^2 + \|\mathbf{x}_3 - \hat{\mathbf{x}}_3\|_2^2 + \|\mathbf{x}_4 - \hat{\mathbf{x}}_4\|_2^2) = \frac{1}{4} (1 + 4 + 0 + 9) = \frac{14}{4} = 3.5$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from matplotlib.collections import LineCollection
X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); N = len(X); fig, ax = plt.subplots(figsize=(3, 3))
ax.set_aspect("equal"); plt.axis([-1.5, 3.5, -1.5, 3.5]); plt.grid(True)
plt.scatter(*X.T, facecolor='white', edgecolor='k', s=100)
K = 1; W = np.array([1, 0]).reshape(-1, K); Z = (X @ W).reshape(-1, K); hX = Z @ W.T
L = np.square(X - hX).sum(axis=1).mean(); ax.set_title(f'$L(W)={L}$')
plt.scatter(*hX.T, facecolor='black', s=100, marker='x')
lines = np.zeros((N, 2, 2)); lines[:, 0, :] = X; lines[:, 1, :] = hX
ax.add_collection(LineCollection(lines, linewidths=2, colors='black', linestyle='dotted'));
```

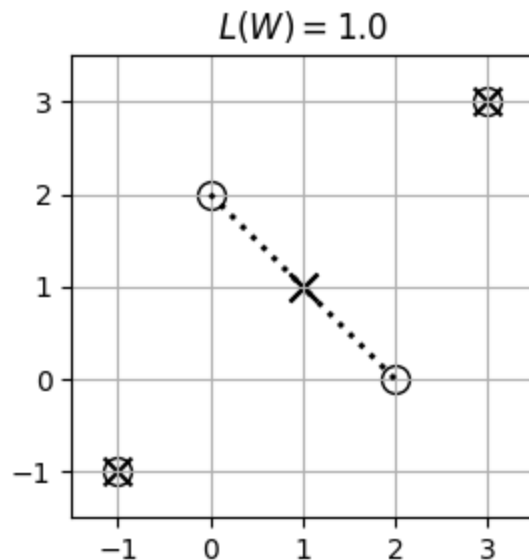


$$\mathbf{W} = (\sqrt{2}/2, \sqrt{2}/2)^t:$$

$$\begin{array}{llll} \mathbf{x}_1 = (-1, -1)^t & \mathbf{x}_2 = (0, 2)^t & \mathbf{x}_3 = (2, 0)^t & \mathbf{x}_4 = (3, 3)^t \\ z_1 = \mathbf{W}^t \mathbf{x}_1 = -\sqrt{2} & z_2 = \sqrt{2} & z_3 = \sqrt{2} & z_4 = 3\sqrt{2} \\ \hat{\mathbf{x}}_1 = \mathbf{W} z_1 = (-1, -1)^t & \hat{\mathbf{x}}_2 = (1, 1)^t & \hat{\mathbf{x}}_3 = (1, 1)^t & \hat{\mathbf{x}}_4 = (3, 3)^t \end{array}$$

$$\mathcal{L}(\mathbf{W}) = \frac{1}{4} (\|\mathbf{x}_1 - \hat{\mathbf{x}}_1\|_2^2 + \|\mathbf{x}_2 - \hat{\mathbf{x}}_2\|_2^2 + \|\mathbf{x}_3 - \hat{\mathbf{x}}_3\|_2^2 + \|\mathbf{x}_4 - \hat{\mathbf{x}}_4\|_2^2) = \frac{1}{4} (0 + 2 + 2 + 0) = 1$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from matplotlib.collections import LineCollection
X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); N = len(X); fig, ax = plt.subplots(figsize=(3, 3))
ax.set_aspect("equal"); plt.axis([-1.5, 3.5, -1.5, 3.5]); plt.grid(True)
plt.scatter(*X.T, facecolor='white', edgecolor='k', s=100)
K = 1; W = np.array([np.sqrt(2)/2, np.sqrt(2)/2]).reshape(-1, K); Z = (X @ W).reshape(-1, K); hX = Z @ W.T
L = np.square(X - hX).sum(axis=1).mean(); ax.set_title(f'$L(W)={L}$')
plt.scatter(*hX.T, facecolor='black', s=100, marker='x')
lines = np.zeros((N, 2, 2)); lines[:, 0, :] = X; lines[:, 1, :] = hX
ax.add_collection(LineCollection(lines, linewidths=2, colors='black', linestyle='dotted'));
```



Cálculo de componentes principales: columnas de la proyección lineal ortogonal $\mathbf{W} \in \mathbb{R}^{D \times K}$ de mínima distorsión

- **Descomposición propia de la matriz de covarianzas empírica ordenada por λ_s en orden no creciente**

$$\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^t \quad \text{donde} \quad \mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_D) \quad \text{y} \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n) \quad \text{con} \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$$

- **Minimizar la distorsión equivale a maximizar la varianza (de los datos proyectados)**
 - $\mathbf{u}_1$ es una dirección de proyección que maximiza la varianza de los datos proyectados, siendo λ_1 dicha varianza
 - Entre todas las direcciones ortonormales a $\mathbf{u}_1$, $\mathbf{u}_2$ y λ_2 son dirección óptima de proyección y varianza asociada
 - Y así, sucesivamente, hasta $\mathbf{u}_K$ y λ_K
- **Proyección lineal ortogonal en $\mathbb{R}^K$ de mínima distorsión o máxima varianza retenida**

$$\mathbf{W}_{\text{pca}} = (\mathbf{u}_1, \dots, \mathbf{u}_K) \in \mathbb{R}^{D \times K}$$

Ejemplo: con los 4 datos 2d, $\mathbf{\Sigma} = [5/2, 3/2; 3/2, 5/2]$ y $\mathbf{W}_{\text{pca}} = \mathbf{u}_1 = (\sqrt{2}/2, \sqrt{2}/2)^t$

```
In [ ]: import numpy as np; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); S = np.cov(X.T, bias=True)
La, U = np.linalg.eigh(S); i = La.argsort()[::-1]; La = La[i]; U = U[:,i]; print(U[:, 0])
[0.70710678 0.70710678]
```

Centrado previo de datos: se suele hacer, aunque $\mathbf{\Sigma}$ (PCA) es invariante a traslaciones de los datos

Elección de K : si se tiene $\mathbf{\Lambda}$, puede escogerse el menor K que explique un porcentaje dado de la varianza total

$$q_K = \frac{1}{\text{tr}(\mathbf{\Sigma})} \sum_{k=1}^K \lambda_k \quad \text{con} \quad \text{tr}(\mathbf{\Sigma}) = \sum_{d=1}^D \lambda_d$$

Ejemplo: con los 4 datos 2d, $q_1 = 4/5 = 80\%$ ya que $\lambda_1 = 4$ y $\lambda_2 = 1$

PCA con la SVD: no requiere calcular la matriz de covarianzas empírica

- **Matriz de datos centrada:** $\mathbf{X} \in \mathbb{R}^{N \times D}$
- **SVD K -truncada de $\mathbf{X}$:** $\hat{\mathbf{X}}_K = \mathbf{U}_K \mathbf{S}_K \mathbf{V}_K^t$
- **Matriz de covarianzas empírica:** $\Sigma = \frac{1}{N} \mathbf{X}^t \mathbf{X}$ (simétrica y semi-definida positiva)
- **SVD K -truncada de Σ :** $\hat{\Sigma}_K = \mathbf{E}_K \mathbf{\Lambda}_K \mathbf{E}_K^t$ con $\mathbf{E}_K = \mathbf{V}_K$ y $\mathbf{\Lambda}_K = \frac{1}{N} \mathbf{S}_K^2$
- **PCA de $\mathbf{X}$ con la SVD:** $\text{PCA}(\mathbf{X}) = \mathbf{X} \mathbf{E}_K \approx \mathbf{U}_K \mathbf{S}_K \mathbf{V}_K^t \mathbf{V}_K = \mathbf{U}_K \mathbf{S}_K$
- **Reconstrucción de $\mathbf{X}$ tras PCA:** $\hat{\mathbf{X}}_K = \text{PCA}(\mathbf{X}) \mathbf{V}_K^t$

```
In [ ]: import numpy as np; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); X = X - np.mean(X, axis=0)
U, S, Vt = np.linalg.svd(X); Xr = U[:, 0] * S[0]
print("Datos centrados:\n", X, "\nComponente principal 1:\n", Vt[:, 0], "\nDatos reducidos:\n", Xr)
print("Datos reconstruidos:\n", Xr.reshape(-1, 1) @ Vt[0, :].reshape(1, -1))
```

Datos centrados:

```
[[-2. -2.]
 [-1.  1.]
 [ 1. -1.]
 [ 2.  2.]]
```

Componente principal 1:

```
[0.70710678 0.70710678]
```

Datos reducidos:

```
[-2.82842712e+00  4.31373875e-16 -3.41684101e-17  2.82842712e+00]
```

Datos reconstruidos:

```
[[-2.00000000e+00 -2.00000000e+00]
 [ 3.05027392e-16  3.05027392e-16]
 [-2.41607145e-17 -2.41607145e-17]
 [ 2.00000000e+00  2.00000000e+00]]
```

3 Gradiente

Función de $\mathbb{R}^n$ en $\mathbb{R}$: $f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad y = f(\mathbf{x})$

Derivada parcial: $\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$ ($\mathbf{e}_i$ es el i-ésimo vector unitario)

Gradiente de f : vector de derivadas parciales

$$\mathbf{g} = \frac{\partial f}{\partial \mathbf{x}} = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \quad \text{o} \quad \mathbf{g}(\mathbf{x}^*) = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}^*} \quad (\text{enfatisa que se evalúa en } \mathbf{x}^*)$$

Neg-gradiente: gradiente con signo cambiado; indica la dirección en la que f decrece más rápidamente

Punto estacionario: punto de gradiente nulo

Hessiana: matriz de derivadas parciales segundas

$$\mathbf{H}(\mathbf{x}) = \frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial g_n}{\partial \mathbf{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial g_n}{\partial x_1} & \cdots & \frac{\partial g_n}{\partial x_n} \end{pmatrix}$$

Tipos de puntos estacionarios: mínimo si la Hessiana es definida positiva; máximo si es definida negativa

Ejercicio: Halla el gradiente de $y = f(\mathbf{x}) = a_1 x_1^2 + a_2 x_2^2$, $a_1, a_2 \neq 0$, puntos estacionarios y tipo de cada uno

Solución: $\mathbf{x} = (x_1, x_2)^t$, $y = f(\mathbf{x}) = a_1 x_1^2 + a_2 x_2^2$, $a_1, a_2 \neq 0$

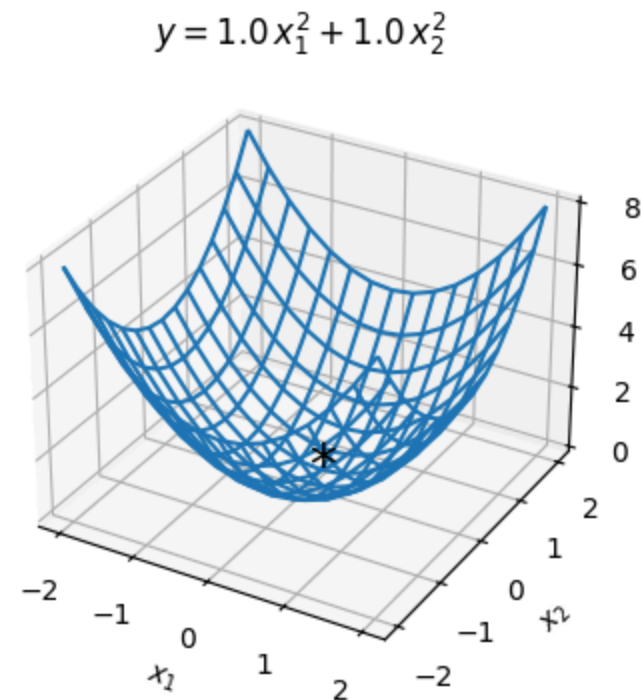
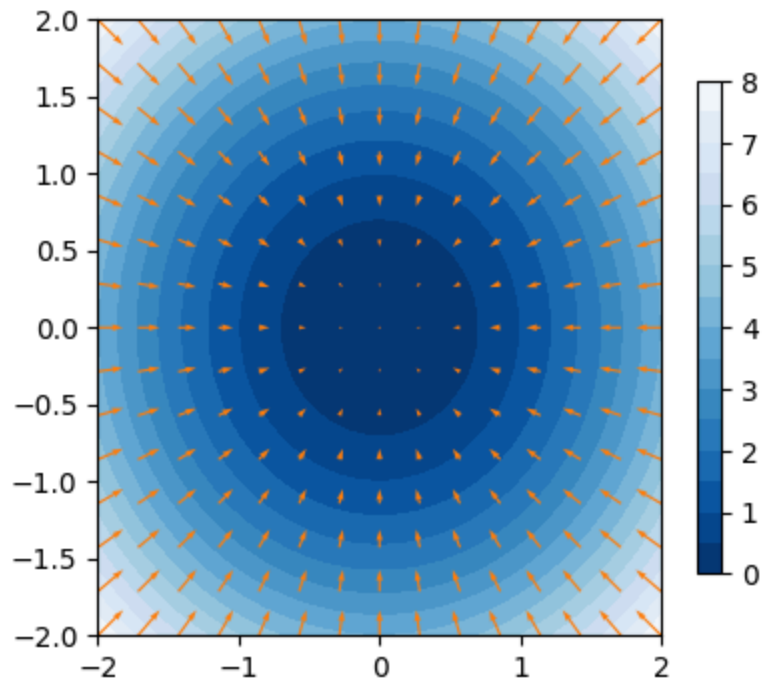
$$\mathbf{g}(\mathbf{x}) = \frac{\partial y}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 2a_1 x_1 \\ 2a_2 x_2 \end{pmatrix}$$
$$\mathbf{g}(\mathbf{x}) = \mathbf{0} \rightarrow \mathbf{x} = \mathbf{0}$$

$$\mathbf{H}(\mathbf{x}) = \frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \mathbf{x}} \\ \frac{\partial g_2}{\partial \mathbf{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} \end{pmatrix}$$
$$= \text{diag}(2a_1, 2a_2) = \begin{cases} \succ 0 & \text{si } a_1, a_2 > 0 \\ \prec 0 & \text{si } a_1, a_2 < 0 \\ \text{indefinida} & a_1, a_2 \text{ de signo opuesto} \end{cases}$$

$$\mathbf{x} = \mathbf{0} \rightarrow \begin{cases} \text{mínimo} & \text{si } a_1, a_2 > 0 \\ \text{máximo} & \text{si } a_1, a_2 < 0 \\ \text{punto de silla} & \text{si } a_1, a_2 \text{ de signo opuesto} \end{cases}$$

Solución (cont.): ejemplo con a_1 y a_2 positivos; prueba ambos negativos y también de signo opuesto

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
R = np.linspace(-2, 2, 50); X1, X2 = np.meshgrid(R, R); X = np.c_[np.ravel(X1), np.ravel(X2)]
a = np.array([1, 1]); Y = (np.square(X) @ a).reshape(X1.shape)
fig = plt.figure(figsize=(10, 4)); ax = plt.subplot(121)
cp = ax.contourf(X1, X2, Y, 15, cmap='Blues_r'); plt.colorbar(cp, ax=ax, shrink=.8)
r = np.linspace(-2, 2, 15); x1, x2 = np.meshgrid(r, r); x = np.c_[np.ravel(x1), np.ravel(x2)]
neg_grad = -x @ np.diag(2 * a); ax.quiver(x[:, 0], x[:, 1], neg_grad[:, 0], neg_grad[:, 1], color='C1');
ax = plt.subplot(122, projection='3d'); ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_title(f'$y={a[0]:.1f}x_1^2 + {a[1]:.1f}x_2^2$'); ax.text(0, 0, 0, '*', fontsize=20)
ax.plot_wireframe(x1, x2, a[0]*x1**2 + a[1]*x2**2);
```



4 Jacobiana

Función de $\mathbb{R}^n$ en $\mathbb{R}^m$: $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $\mathbf{y} = \mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^t$

Jacobiana de $\mathbf{f}$: matriz de derivadas parciales

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial f_m}{\partial \mathbf{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^t \\ \vdots \\ \nabla f_m(\mathbf{x})^t \end{pmatrix}$$

Jacobiana de una función de $\mathbb{R}^n$ en $\mathbb{R}$: es su gradiente transpuesto

Jacobiana del gradiente de una función de $\mathbb{R}^n$ en $\mathbb{R}$: es su Hessiana

$$\mathbf{J} = \frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial g_n}{\partial \mathbf{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial g_n}{\partial x_1} & \cdots & \frac{\partial g_n}{\partial x_n} \end{pmatrix} = \mathbf{H}(\mathbf{x})$$

Ejercicio: Halla la Jacobiana de $\mathbf{y} = \mathbf{W}\mathbf{x}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{W} \in \mathbb{R}^{m \times n}$, $m = 2$, $n = 3$

Solución: $\mathbf{y} = \mathbf{W}\mathbf{x}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{W} \in \mathbb{R}^{m \times n}$, $m = 2$, $n = 3$

$$\begin{aligned} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} &= \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \\ &= \begin{pmatrix} W_{11}x_1 + W_{12}x_2 + W_{13}x_3 \\ W_{21}x_1 + W_{22}x_2 + W_{23}x_3 \end{pmatrix} \end{aligned}$$

$$\mathbf{J} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{x}} \\ \frac{\partial y_2}{\partial \mathbf{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \frac{\partial y_1}{\partial x_3} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_3} \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix} = \mathbf{W}$$

5 Descenso por gradiente

Objetivo: minimizar $f : \mathbb{R}^n \rightarrow \mathbb{R}$ iterativamente, a partir de un punto de inicio $\mathbf{x}_0$

Descenso por gradiente (GD): escoge la dirección de descenso más pronunciada, dada por el neg-gradiente

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \eta_i \mathbf{g}(\mathbf{x}_i) \quad \text{donde} \quad \eta_i > 0 \quad (i = 0, 1, \dots)$$

Factor de aprendizaje constante: $\eta_i = \eta$; GD puede no converger o hacerlo muy lentamente

Caída escalonada: $\eta_i = \eta_0 \gamma^{\lfloor i/s \rfloor}$ con *decay rate* γ y *decay step* s ; por ejemplo, $\eta_0 = 0.1$, $\gamma = 0.5$ y $s = 10$

Momentum: acelera el movimiento en regiones llanas del objetivo (y lo frena en abruptas)

$$\begin{aligned} \mathbf{x}_{i+1} &= \mathbf{x}_i - \eta \mathbf{m}_{i+1} \\ \mathbf{m}_{i+1} &= \beta \mathbf{m}_i + \mathbf{g}(\mathbf{x}_i) \quad \text{donde} \quad \beta \in [0, 1) \quad (\text{típicamente } \beta = 0.9) \end{aligned}$$

Inconveniente: momentum suele oscilar al acercarse a un mínimo

Momentum Nesterov: añade un paso de extrapolación para amortiguar oscilaciones

$$\begin{aligned} \mathbf{x}_{i+1} &= \mathbf{x}_i + \mathbf{m}_{i+1} \\ \mathbf{m}_{i+1} &= \beta \mathbf{m}_i - \eta \mathbf{g}(\mathbf{x}_i + \beta \mathbf{m}_i) \quad \text{donde} \quad \beta \in [0, 1) \quad (\text{típicamente } \beta = 0.9) \end{aligned}$$

Implementación: GD con caída escalonada y momentum Nesterov; devuelve el camino recorrido

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from matplotlib.collections import LineCollection
```

```
In [ ]: def GD(g, x0, eta=0.1, decay_rate=0.5, decay_step=10, beta=0.9, tol=1e-3, max_iter=100000):
    n = len(x0); X = np.zeros((max_iter + 1, n)); X[0, :] = x0
    delta = np.inf; i = 0; m = np.zeros(2)
    while np.max(np.abs(delta)) > tol and i < max_iter:
        m = beta * m - eta * g(X[i, :] + beta * m)
        delta = m
        X[i+1, :] = X[i, :] + delta
        i = i + 1
        if i % decay_step == 0: eta = eta * decay_rate
    return X[i+1, :]
```

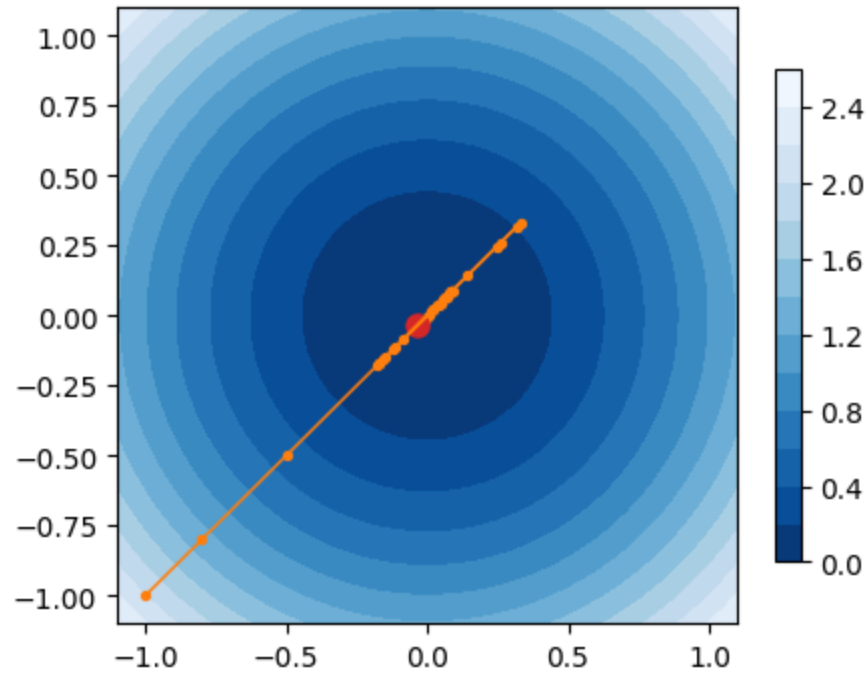
Función auxiliar: para mostrar el camino recorrido por GD en 2d

```
In [ ]: def plot_GD(f, P, figsize=(5, 4), xlim=(-1.1, 1.1), ylim=(-1.1, 1.1), levels=15, cmap='Blues_r'):
    X1, X2 = np.meshgrid(np.linspace(*xlim, 50), np.linspace(*ylim, 50))
    X = np.c_[np.ravel(X1), np.ravel(X2)]
    Y = np.apply_along_axis(f, 1, X).reshape(X1.shape)
    _, ax = plt.subplots(1, 1, figsize=figsize)
    ax.set_xlim(xlim); ax.set_ylim(ylim)
    cp = ax.contourf(X1, X2, Y, levels=levels, cmap=cmap)
    plt.colorbar(cp, ax=ax, shrink=.8)
    ax.scatter(*P.T, c='C1', s=8)
    ax.scatter(*P[-1, :].T, c='C3', s=64)
    L = np.zeros((len(P)-1, 2, 2))
    L[:, 0, :] = P[:-1, :]; L[:, 1, :] = P[1:, :]
    ax.add_collection(LineCollection(L, lw=1, colors='C1', ls='solid'));
```

Ejemplo (cont.): Minimización de $y = a_1x_1^2 + a_2x_2^2$, $a_1 = a_2 = 1$, a partir de $\mathbf{x}_0 = (-1, -1)^t$

```
In [ ]: a = np.array([1, 1]); f = lambda x: (a @ np.square(x)).sum(); g = lambda x: x @ np.diag(2 * a)
P = GD(g, np.array([-1, -1])); plot_GD(f, P)
print(f'iteraciones = {len(P)-1} solución = {np.round(P[-1, :], 4)}')
```

iteraciones = 45 solución = [-0.0337 -0.0337]



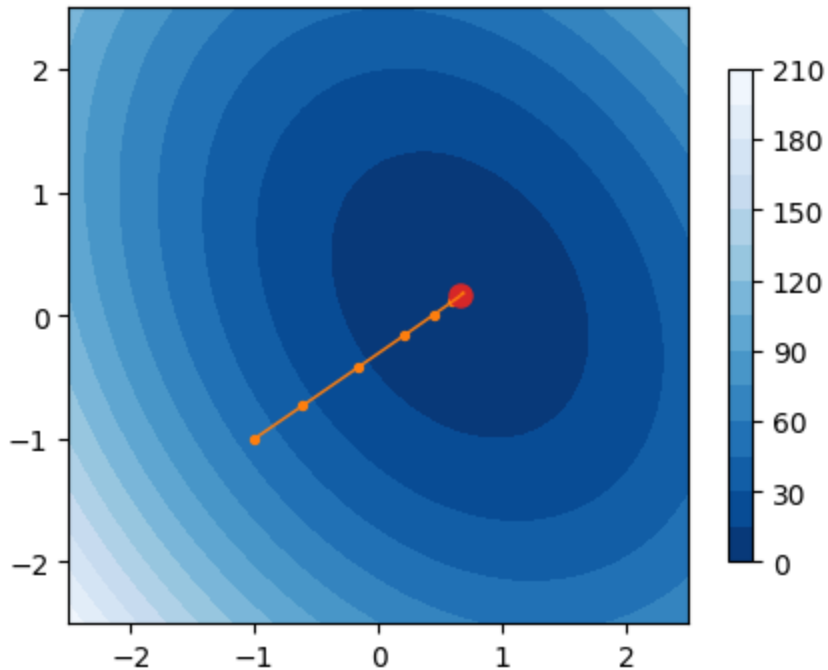
Ejemplo: número de iteraciones de GD con una función cuadrática de matriz bien condicionada

$$y = \frac{1}{2} \mathbf{x}^t \mathbf{A} \mathbf{x} + \mathbf{b}^t \mathbf{x} + c, \quad \mathbf{A} = \begin{pmatrix} 20 & 5 \\ 5 & 16 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} -14 \\ -6 \end{pmatrix}, \quad c = 10, \quad \mathbf{x}^* = -2(\mathbf{A} + \mathbf{A}^t)^{-1} \mathbf{b}$$

$$\mathbf{x}_0 = (-1, -1)^t, \quad \mathbf{g}(\mathbf{x}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^t) \mathbf{x} + \mathbf{b}$$

```
In [ ]: A, b, c = np.array([ [20, 5], [5, 16] ]), np.array([-14, -6]), 10; xstar = -2 * np.linalg.inv(A+A.T) @ b
f = lambda x: 0.5 * x.T @ A @ x + b @ x + c; g = lambda x: 0.5 * (A + A.T) @ x + b
P = GD(g, np.array([-1, -1]), eta=0.01, beta=0.5, tol=np.finfo(float).eps)
plot_GD(f, P, xlim=(-2.5, 2.5), ylim=(-2.5, 2.5))
print(f'min = {np.round(xstar, 4)} i = {len(P)-1} sol = {np.round(P[-1, :], 4)}')
```

```
min = [0.6576 0.1695] i = 361 sol = [0.6577 0.1695]
```



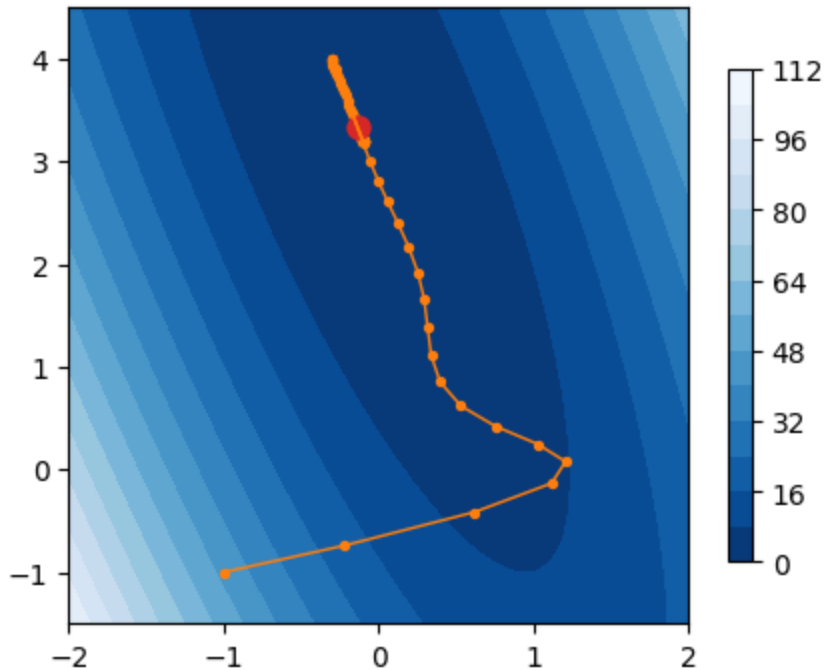
Ejemplo (cont.): número de iteraciones de GD con una función cuadrática de matriz peor condicionada

$$y = \frac{1}{2} \mathbf{x}^t \mathbf{A} \mathbf{x} + \mathbf{b}^t \mathbf{x} + c, \quad \mathbf{A} = \begin{pmatrix} 20 & 5 \\ 5 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} -14 \\ -6 \end{pmatrix}, \quad c = 10, \quad \mathbf{x}^* = -2(\mathbf{A} + \mathbf{A}^t)^{-1} \mathbf{b}$$

$$\mathbf{x}_0 = (-1, -1)^t, \quad \mathbf{g}(\mathbf{x}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^t) \mathbf{x} + \mathbf{b}$$

```
In [ ]: A, b, c = np.array([ [20, 5], [5, 2] ]), np.array([-14, -6]), 10; xstar = -2 * np.linalg.inv(A+A.T) @ b
f = lambda x: 0.5 * x.T @ A @ x + b @ x + c; g = lambda x: 0.5 * (A + A.T) @ x + b
P = GD(g, np.array([-1, -1]), eta=0.02, decay_step=50, beta=0.9, tol=np.finfo(float).eps)
plot_GD(f, P, xlim=(-2, 2), ylim=(-1.5, 4.5))
print(f'min = {np.round(xstar, 4)} i = {len(P)-1} sol = {np.round(P[-1, :], 4)}')
```

```
min = [-0.1333  3.3333] i = 1867 sol = [-0.1334  3.3335]
```



Diferenciación simbólica: diferenciación mediante computación simbólica

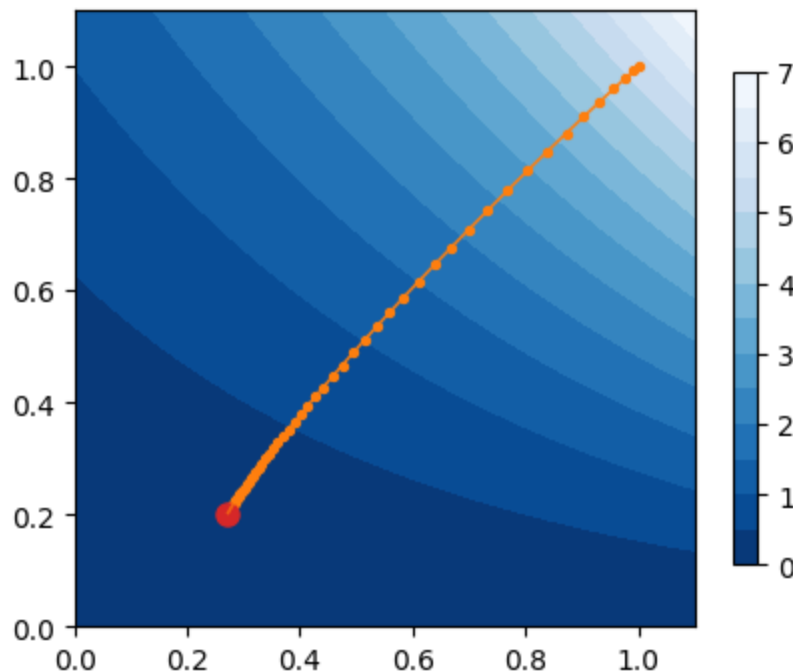
SymPy: librería python de computación simbólica

Ejemplo: $f(x_1, x_2) = x_2 \exp(x_1) \sqrt{x_1 + x_2 \exp(x_1)}$

```
In [ ]: from sympy import symbols, exp, sqrt, simplify, lambdify
x, y = symbols('x1 x2')
spf = y * sqrt(x + y * exp(x)) * exp(x); f = lambdify([x, y], spf, "numpy")
spdx = simplify(spf.diff(x)); dx = lambdify([x, y], spdx, "numpy")
spdy = simplify(spf.diff(y)); dy = lambdify([x, y], spdy, "numpy") # display(spf, spdx, spdy)
```

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
npf = lambda xy: f(xy[0], xy[1])
npg = lambda xy: np.array([dx(xy[0], xy[1]), dy(xy[0], xy[1])])
P = GD(npg, np.array([1, 1]), eta=.001); plot_GD(npf, P, xlim=(0, 1.1), ylim=(0, 1.1))
print(f'iteraciones = {len(P)-1} solución = {np.round(P[-1, :], 4)}')
```

iteraciones = 61 solución = [0.2712 0.2006]



Función (valle o banana de) Rosenbrock: mínimo global en (a, a^2)

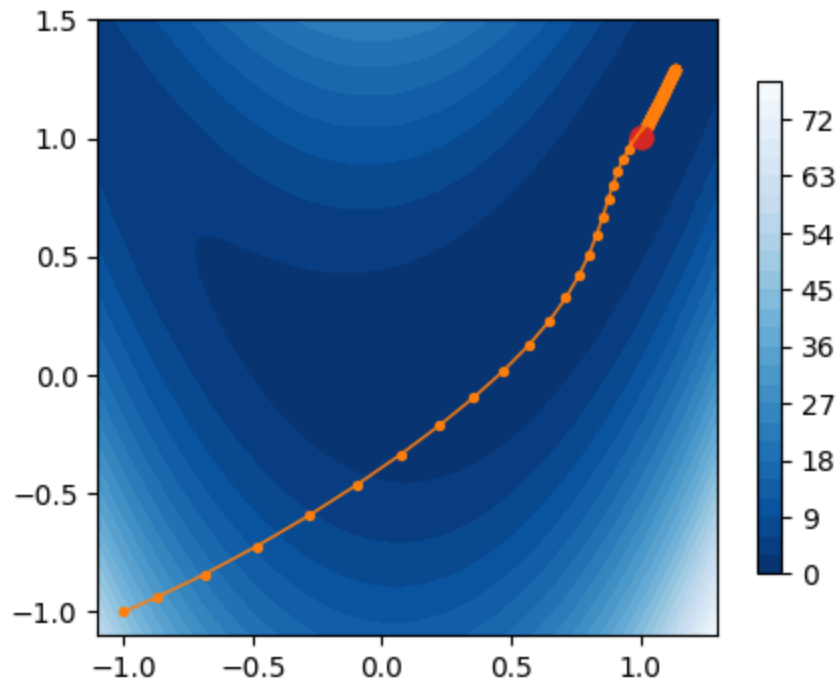
$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

Ejemplo: GD aplicado a Rosenbrock con $a = 1$, $b = 10$ y punto de inicio $(-1, -1)^t$

```
In [ ]: from sympy import symbols, simplify, lambdify
x, y = symbols('x1 x2')
spf = (1-x)**2 + 10 * (y-x**2)**2; f = lambdify([x, y], spf, "numpy")
spdx = simplify(spf.diff(x)); dx = lambdify([x, y], spdx, "numpy")
spdy = simplify(spf.diff(y)); dy = lambdify([x, y], spdy, "numpy") # display(spf, spdx, spdy)
```

```
In [ ]: npf = lambda xy: f(xy[0], xy[1]); npg = lambda xy: np.array([dx(xy[0], xy[1]), dy(xy[0], xy[1])])
tol = np.finfo(float).eps
P = GD(npg, np.array([-1, -1]), eta=0.0015, decay_step=100, decay_rate=0.9, beta=0.9, tol=np.finfo(float).eps)
plot_GD(npf, P, xlim=(-1.1, 1.3), ylim=(-1.1, 1.5), levels=30, cmap='Blues_r')
print(f'i = {len(P)-1}   sol = {np.round(P[-1, :], 4)}')
```

i = 22918 sol = [1.0005 1.0011]



6 Cálculo matricial

Cálculo matricial: notación especializada para manejar derivadas parciales

Formato numerador: primero el del numerador y luego el del denominador transpuesto, **si cabe en una matriz**

Tipos	Escalar $y \in \mathbb{R}^{1 \times 1}$	Vector $\mathbf{y} \in \mathbb{R}^{m \times 1}$	Matriz $\mathbf{Y} \in \mathbb{R}^{m \times n}$
$x \in \mathbb{R}^{1 \times 1}$	$\frac{\partial y}{\partial x}$	$\frac{\partial \mathbf{y}}{\partial x} = \begin{bmatrix} \frac{\partial y_1}{\partial x} \\ \vdots \\ \frac{\partial y_m}{\partial x} \end{bmatrix}$	$\frac{\partial \mathbf{Y}}{\partial x} = \begin{bmatrix} \frac{\partial y_{11}}{\partial x} & \dots & \frac{\partial y_{1n}}{\partial x} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_{m1}}{\partial x} & \dots & \frac{\partial y_{mn}}{\partial x} \end{bmatrix}$
$\mathbf{x} \in \mathbb{R}^{n \times 1}$	$\frac{\partial y}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \dots & \frac{\partial y}{\partial x_n} \end{bmatrix}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial y_m}{\partial \mathbf{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$	$\frac{\partial \mathbf{Y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_{11}}{\partial \mathbf{x}} & \dots & \frac{\partial y_{1n}}{\partial \mathbf{x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_{m1}}{\partial \mathbf{x}} & \dots & \frac{\partial y_{mn}}{\partial \mathbf{x}} \end{bmatrix} \quad 3\text{T}$
$\mathbf{X} \in \mathbb{R}^{p \times q}$	$\frac{\partial y}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial y}{\partial x_{11}} & \dots & \frac{\partial y}{\partial x_{p1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y}{\partial x_{1q}} & \dots & \frac{\partial y}{\partial x_{pq}} \end{bmatrix}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{X}} \\ \vdots \\ \frac{\partial y_m}{\partial \mathbf{X}} \end{bmatrix} \quad 3\text{T}$	$\frac{\partial \mathbf{Y}}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial y_{11}}{\partial \mathbf{X}} & \dots & \frac{\partial y_{1n}}{\partial \mathbf{X}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_{m1}}{\partial \mathbf{X}} & \dots & \frac{\partial y_{mn}}{\partial \mathbf{X}} \end{bmatrix} \quad 4\text{T}$

Casos de interés: los de la columna izquierda y centro . . . qué hacemos con el 3-tensor $\frac{\partial \mathbf{y}}{\partial \mathbf{X}} \in \mathbb{R}^{m \times q \times p}$?

Caso de interés especial:

- En general no necesitaremos hacer cálculos con el 3-tensor $\frac{\partial \mathbf{y}}{\partial \mathbf{X}} \in \mathbb{R}^{m \times q \times p}$
- Sí necesitaremos calcular $\mathbf{u}^t \frac{\partial \mathbf{y}}{\partial \mathbf{X}}$ con $\mathbf{u}^t \in \mathbb{R}^{1 \times m}$ constante
- $\mathbf{u}^t \frac{\partial \mathbf{y}}{\partial \mathbf{X}} \in \mathbb{R}^{1 \times q \times p}$ con un único elemento $\mathbb{R}^{q \times p}$ en la primera dimensión, que extraemos por simplicidad

$$\mathbf{u}^t \frac{\partial \mathbf{y}}{\partial \mathbf{X}} = [u_1, \dots, u_m] \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{X}} \\ \vdots \\ \frac{\partial y_m}{\partial \mathbf{X}} \end{bmatrix} = \left[\sum_{i=1}^m u_i \frac{\partial y_i}{\partial \mathbf{X}} \right] \rightarrow \left[\mathbf{u}^t \frac{\partial \mathbf{y}}{\partial \mathbf{X}} \right]_{1,:} = \sum_{i=1}^m u_i \frac{\partial y_i}{\partial \mathbf{X}}$$

Formato denominador: primero el del denominador y luego el del numerador transpuesto, **si cabe en una matriz**

- El resultado es el mismo que en formato numerador, aunque transpuesto

Origen de los dos formatos: se halla, sobre todo, en el formato que se da a $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$

Formato mixto: consiste en escribir $\frac{\partial \mathbf{y}}{\partial \mathbf{x}^t}$ y seguir el formato numerador

Recomendación para evitar errores: usar siempre el mismo formato, numerador si puede ser

Identidades para escalares: a constante; u y v funciones reales de $x \in \mathbb{R}^{1 \times 1}$, $\mathbf{x} \in \mathbb{R}^{n \times 1}$ o $\mathbf{X} \in \mathbb{R}^{p \times q}$

$y \in \mathbb{R}^{1 \times 1}$	$\frac{\partial y}{\partial x} \in \mathbb{R}^{1 \times 1}$	$\frac{\partial y}{\partial \mathbf{x}} \in \mathbb{R}^{1 \times n}$	$\frac{\partial y}{\partial \mathbf{X}} \in \mathbb{R}^{q \times p}$
a	0	$\mathbf{0}$	$\mathbf{0}$
au	$a \frac{\partial u}{\partial x}$	$a \frac{\partial u}{\partial \mathbf{x}}$	$a \frac{\partial u}{\partial \mathbf{X}}$
$u + v$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial x}$	$\frac{\partial u}{\partial \mathbf{x}} + \frac{\partial v}{\partial \mathbf{x}}$	$\frac{\partial u}{\partial \mathbf{X}} + \frac{\partial v}{\partial \mathbf{X}}$
uv	$u \frac{\partial v}{\partial x} + v \frac{\partial u}{\partial x}$	$u \frac{\partial v}{\partial \mathbf{x}} + v \frac{\partial u}{\partial \mathbf{x}}$	$u \frac{\partial v}{\partial \mathbf{X}} + v \frac{\partial u}{\partial \mathbf{X}}$
$g(u)$	$\frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial x}$	$\frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial \mathbf{x}}$	$\frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial \mathbf{X}}$
$f(g(u))$	$\frac{\partial f(g)}{\partial g} \frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial x}$	$\frac{\partial f(g)}{\partial g} \frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial \mathbf{x}}$	$\frac{\partial f(g)}{\partial g} \frac{\partial g(u)}{\partial u} \frac{\partial u}{\partial \mathbf{X}}$

Identidades para vectores: $\mathbf{a}$ constante; $\mathbf{u}$ y $\mathbf{v}$ funciones vectoriales de $x \in \mathbb{R}^{1 \times 1}$ o $\mathbf{x} \in \mathbb{R}^{n \times 1}$

$\mathbf{y} \in \mathbb{R}^{m \times 1}$	$\frac{\partial \mathbf{y}}{\partial x} \in \mathbb{R}^{m \times 1}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$
$\mathbf{a}$	$\mathbf{0}$	$\mathbf{0}$
$a\mathbf{u}$	$a \frac{\partial \mathbf{u}}{\partial x}$	$a \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$
$\mathbf{u} + \mathbf{v}$	$\frac{\partial \mathbf{u}}{\partial x} + \frac{\partial \mathbf{v}}{\partial x}$	$\frac{\partial \mathbf{u}}{\partial \mathbf{x}} + \frac{\partial \mathbf{v}}{\partial \mathbf{x}}$
$\mathbf{g}(\mathbf{u})$	$\frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial x}$	$\frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$
$\mathbf{f}(\mathbf{g}(\mathbf{u}))$	$\frac{\partial \mathbf{f}(\mathbf{g})}{\partial \mathbf{g}} \frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial x}$	$\frac{\partial \mathbf{f}(\mathbf{g})}{\partial \mathbf{g}} \frac{\partial \mathbf{g}(\mathbf{u})}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$

Transformaciones lineales: $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} = \mathbf{W}\mathbf{x}$, $\mathbf{W} \in \mathbb{R}^{m \times n}$, $\mathbf{u} \in \mathbb{R}^m$ constante

$$\frac{\partial y_i}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_k W_{ik} x_k = \sum_k W_{ik} \frac{\partial x_k}{\partial x_j} = W_{ij}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial y_m}{\partial \mathbf{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} = \mathbf{W}$$

$$\frac{\partial y_k}{\partial W_{ij}} = \frac{\partial}{\partial W_{ij}} \sum_l W_{kl} x_l = \sum_l x_l \frac{\partial W_{kl}}{\partial W_{ij}} = x_j \mathbb{I}(k = i)$$

$$\frac{\partial y_k}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial y_k}{\partial W_{11}} & \frac{\partial y_k}{\partial W_{21}} & \cdots & \frac{\partial y_k}{\partial W_{m1}} \\ \frac{\partial y_k}{\partial W_{12}} & \frac{\partial y_k}{\partial W_{22}} & \cdots & \frac{\partial y_k}{\partial W_{m2}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_k}{\partial W_{1n}} & \frac{\partial y_k}{\partial W_{2n}} & \cdots & \frac{\partial y_k}{\partial W_{mn}} \end{bmatrix} = \begin{bmatrix} x_1 \mathbb{I}(k=1) & x_1 \mathbb{I}(k=2) & \cdots & x_1 \mathbb{I}(k=m) \\ x_2 \mathbb{I}(k=1) & x_2 \mathbb{I}(k=2) & \cdots & x_2 \mathbb{I}(k=m) \\ \vdots & \vdots & \ddots & \vdots \\ x_n \mathbb{I}(k=1) & x_n \mathbb{I}(k=2) & \cdots & x_n \mathbb{I}(k=m) \end{bmatrix} = \mathbf{x} \text{onehot}(k)^t$$

$$\mathbf{u}^t \frac{\partial \mathbf{y}}{\partial \mathbf{W}} = \sum_{k=1}^m u_k \frac{\partial y_k}{\partial \mathbf{W}} = \sum_{k=1}^m u_k \mathbf{x} \text{onehot}(k)^t = \mathbf{x} \sum_{k=1}^m u_k \text{onehot}(k)^t = \mathbf{x} \mathbf{u}^t$$

Transformaciones elementales: $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} = \boldsymbol{\varphi}(\mathbf{x}) \in \mathbb{R}^n$, $y_i = \varphi(x_i) \quad (i = 1, \dots, n)$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} \varphi(x_1) \\ \varphi(x_2) \\ \vdots \\ \varphi(x_n) \end{bmatrix}$$

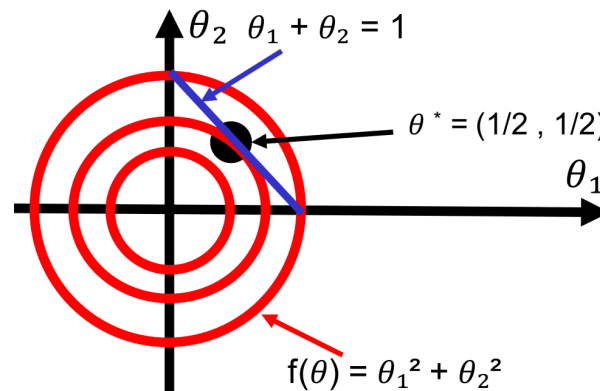
$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial y_n}{\partial \mathbf{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \cdots & \frac{\partial y_n}{\partial x_n} \end{bmatrix} = \text{diag}(\varphi'(x_1), \dots, \varphi'(x_n))$$

7 Optimización con restricciones

Optimización con restricciones: el conjunto de soluciones posibles se restringe con igualdades y desigualdades

$$\theta^* \in \underset{\theta \in \mathcal{C}}{\operatorname{argmin}} \mathcal{L}(\theta) \quad \text{con} \quad \mathcal{C} = \{\theta : g_j(\theta) \leq 0 : j \in \mathcal{I}, h_k(\theta) = 0 : k \in \mathcal{E}\} \in \mathbb{R}^D$$

Ejemplo: Minimizar $\mathcal{L}(\theta) = \theta_1^2 + \theta_2^2$ sujeto a $h(\theta) = 1 - \theta_1 - \theta_2 = 0 \rightarrow \theta^* = (0.5, 0.5)$



Restricciones de igualdad: se "suben" al objetivo junto con **multiplicadores de Lagrange** y se optimiza sin más

$$L(\theta, \lambda) = \mathcal{L}(\theta) + \sum_k \lambda_k h_k(\theta)$$

Ejemplo (cont.): $L(\theta_1, \theta_2, \lambda) = \theta_1^2 + \theta_2^2 + \lambda(\theta_1 + \theta_2 - 1)$

$$\frac{\partial L}{\partial(\theta_1, \theta_2, \lambda)} = \begin{pmatrix} 2\theta_1 + \lambda \\ 2\theta_2 + \lambda \\ \theta_1 + \theta_2 - 1 \end{pmatrix} \Big|_{(\theta_1^*, \theta_2^*, \lambda^*)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow \theta_1^* = \theta_2^* = 0.5$$

Restricciones de igualdad y desigualdad: ignoramos las de desigualdad y se comprueban en la solución

8 Distribución de Bernoulli

Distribución de Bernoulli: $Y \sim \text{Ber}(\theta)$, $\theta \in [0, 1]$, si su pmf $p : \{0, 1\} \rightarrow [0, 1]$ es

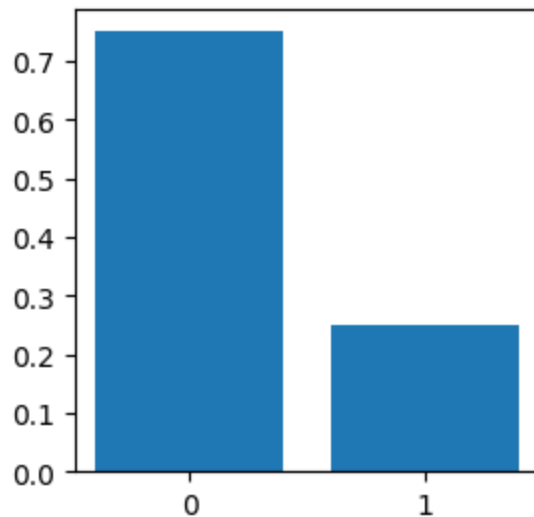
$$p(y | \theta) = \begin{cases} 1 - \theta & \text{si } y = 0 \\ \theta & \text{si } y = 1 \end{cases} = \theta^y + (1 - \theta)(1 - y) = \theta^y(1 - \theta)^{1-y} \quad (0^0 = 1, 0 \log 0 = 0)$$

- Y es el resultado de un **experimento** con probabilidad de **éxito** ($Y = 1$) θ y probabilidad de **fracaso** ($Y = 0$) $1 - \theta$

Ejemplo: $\theta = 0.25$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import bernoulli
t = 0.25; Y = bernoulli(t); print(Y.rvs(10)); y = (0, 1)
fig = plt.subplots(figsize=(3, 3)); plt.xticks(y); plt.bar(y, Y.pmf(y));
```

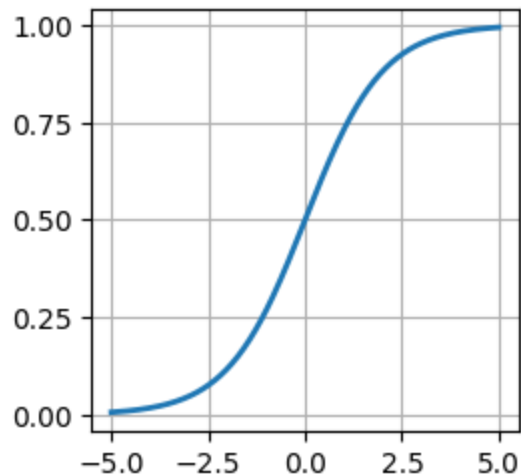
[0 0 0 0 1 0 0 0 0 1]



Función sigmoide: función $\sigma : \mathbb{R} \rightarrow [0, 1]$ con forma de S

$$\sigma(a) = \frac{1}{1 + e^{-a}} = \frac{e^a}{1 + e^a}$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
def sigmoid(a):
    return 1 / (1 + np.exp(-a))
fig = plt.subplots(figsize=(2.75, 2.75)); plt.grid(); plt.yticks(np.arange(0, 1.1, step=0.25))
a = np.linspace(-5, 5, 200); plt.plot(a, sigmoid(a), linewidth=2);
```



Propiedad:

$$1 - \sigma(a) = 1 - \frac{1}{1 + e^{-a}} = \frac{e^{-a}}{1 + e^{-a}} = \frac{1}{1 + e^a} = \sigma(-a)$$

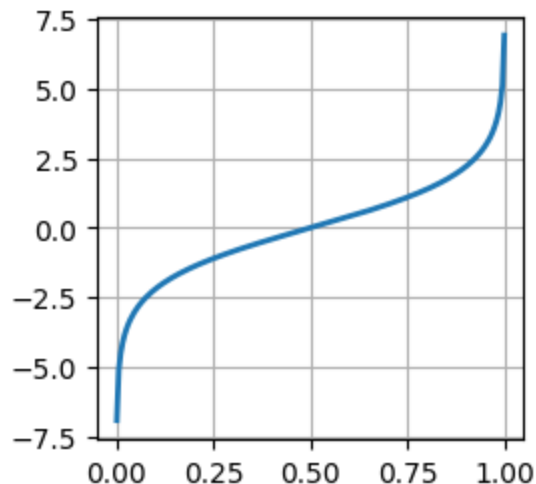
Función logit: función $\text{logit} : [0, 1] \rightarrow \mathbb{R}$ inversa de la sigmoide, $\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$

Interpretación: si p es la probabilidad de que un cierto evento ocurra, su logit es la log-posibilidad (log-odds) de que el evento ocurra frente a que no ocurra, por lo que tenemos tres casos

1. Más posibilidades de ocurrir que de no: odds mayor que 1 y log-odds positiva
2. Igual posibilidades de ocurrir que de no: odds 1 y log-odds nula
3. Menos posibilidades de ocurrir que de no: odds menor que 1 y log-odds negativa

Inversa de la sigmoide: $\text{logit}(\sigma(a)) = \log\left(\frac{\sigma(a)}{1-\sigma(a)}\right) = \log\left(\frac{e^a}{1+e^a} \frac{1+e^a}{1}\right) = \log(e^a) = a$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
def logit(p):
    return np.log(p / (1 - p))
fig = plt.subplots(figsize=(2.75, 2.75)); plt.grid(); plt.xticks(np.arange(0, 1.1, step=0.25))
p = np.linspace(.001, .999, 200); plt.plot(p, logit(p), linewidth=2);
```



Recordatorio: la sigmoide transforma log-odds en probabilidad y la logit probabilidad en log-odds

9 Distribución categórica

Propósito: modelizar una distribución sobre un conjunto finito de etiquetas $\mathcal{C} = \{1, \dots, C\}$, $C \geq 2$

Codificación one-hot: $y \in \mathcal{C}$, $\text{one-hot}(y) = \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_C \end{pmatrix} = \begin{pmatrix} \mathbb{I}(y=1) \\ \vdots \\ \mathbb{I}(y=C) \end{pmatrix} \in \{0, 1\}^C$ con $\sum_c y_c = 1$

Distribución categórica: $Y \sim \text{Cat}(\boldsymbol{\theta})$, $\boldsymbol{\theta} \in [0, 1]^C$, $\sum_c \theta_c = 1$, si su pmf $p : \mathcal{C} \rightarrow [0, 1]$ es

$$p(y \mid \boldsymbol{\theta}) = \prod_{c=1}^C \theta_c^{\mathbb{I}(y=c)} \quad \text{o, en notación one-hot,} \quad p(\mathbf{y} \mid \boldsymbol{\theta}) = \prod_{c=1}^C \theta_c^{y_c}$$

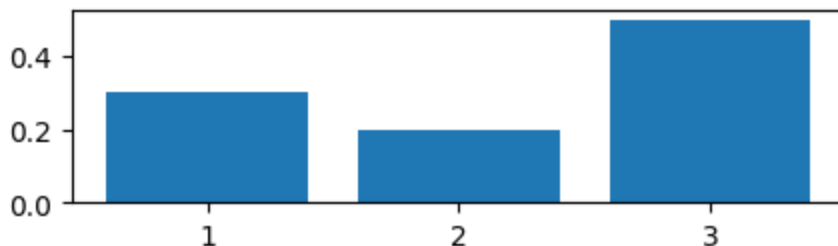
Interpretación: θ_c es la probabilidad de que y valga c , $p(y = c \mid \boldsymbol{\theta}) = \theta_c$ (convenimos $0^0 = 1$ y $0 \log 0 = 0$)

Sobre-parametrización: solo tenemos $C - 1$ parámetros libres por las restricciones sobre $\boldsymbol{\theta}$

Ejemplo: $C = 3$, $\boldsymbol{\theta}^t = (0.3, 0.2, 0.5)$

```
In [ ]: import matplotlib.pyplot as plt; from scipy.stats import multinomial
t = [0.3, 0.2, 0.5]; Y = multinomial(1, t); print(Y.p, Y.rvs(3)[1, :])
fig = plt.subplots(figsize=(5, 1.25)); y = range(1, 4); plt.xticks(y); plt.bar(y, Y.p);
```

[0.3 0.2 0.5] [0 0 1]

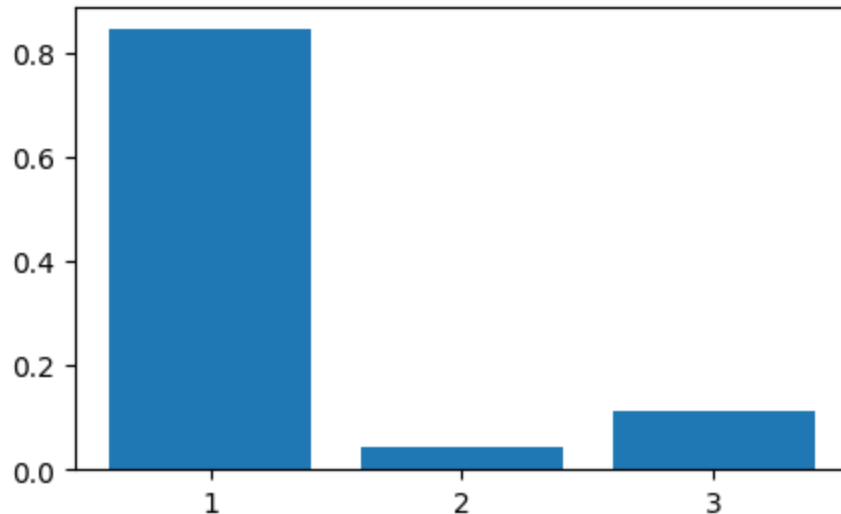


Función softmax: transforma logits $\mathbf{a} \in \mathbb{R}^C$ en un vector de probabilidades $[0, 1]^C$

$$\mathcal{S}(\mathbf{a}) = \left(\frac{e^{a_1}}{\sum_{\tilde{c}} e^{a_{\tilde{c}}}}, \dots, \frac{e^{a_C}}{\sum_{\tilde{c}} e^{a_{\tilde{c}}}} \right)^t \quad \text{cumpliéndose} \quad 0 \leq \mathcal{S}(\mathbf{a})_c \leq 1 \quad \text{y} \quad \sum_{c=1}^C \mathcal{S}(\mathbf{a})_c = 1$$

Ejemplo: $\mathbf{a} = (3, 0, 1)^t$, $\mathcal{S}(\mathbf{a}) = \left(\frac{e^3}{e^3 + 1 + e}, \frac{1}{e^3 + 1 + e}, \frac{e}{e^3 + 1 + e} \right)^t = (0.8438, 0.0420, 0.1142)^t$

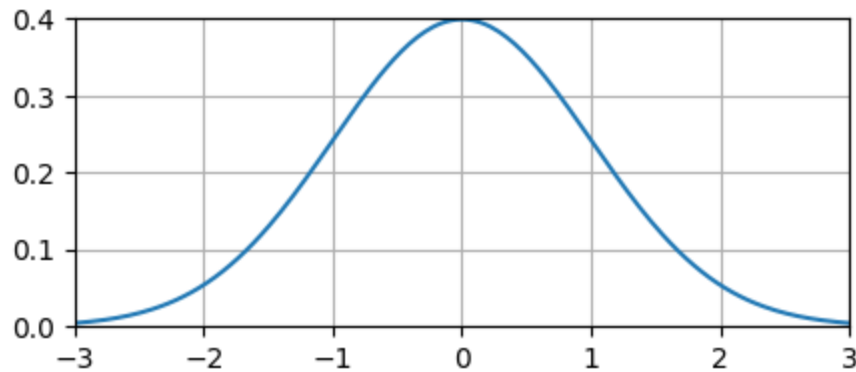
```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
def softmax(a):
    e = np.exp((1.0 * np.array(a))); return e / np.sum(e)
a = np.array([3, 0, 1]); fig = plt.subplots(figsize=(5, 3));
y = range(1, 4); plt.xticks(y); plt.bar(y, softmax(a));
```



10 Distribución Gaussiana

Función de densidad de probabilidad (pdf) Gaussiana: $Y \sim \mathcal{N}(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y - \mu)^2\right)$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import norm
Y = norm(0, 1); y = np.linspace(-3, 3, 200); fig = plt.subplots(figsize=(5, 2))
plt.grid(); plt.xlim(-3, 3); plt.ylim(0, .4); plt.plot(y, Y.pdf(y));
```



Covarianza entre dos variables aleatorias X e Y : mide hasta qué grado X e Y están (linealmente) relacionadas

$$\text{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

Gaussiana multivariada: $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, $\mathbf{x} \in \mathbb{R}^D$, media $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}] \in \mathbb{R}^D$, covarianzas $\boldsymbol{\Sigma} = \text{Cov}[\mathbf{x}] \in \mathbb{R}^{D \times D}$

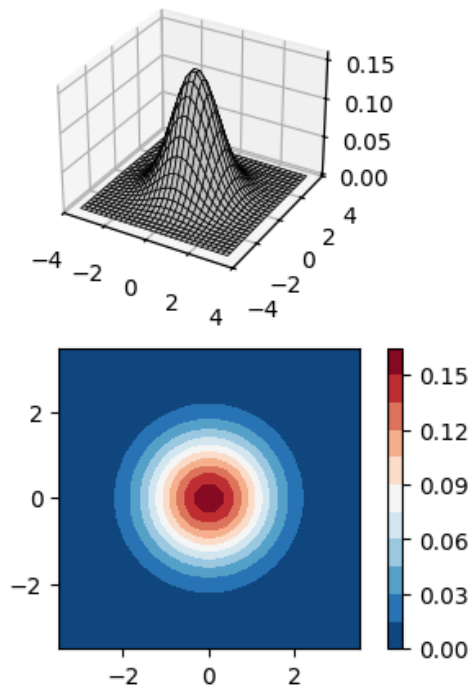
$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^t \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right]$$

Tipos de Gaussianas según estructura de Σ :

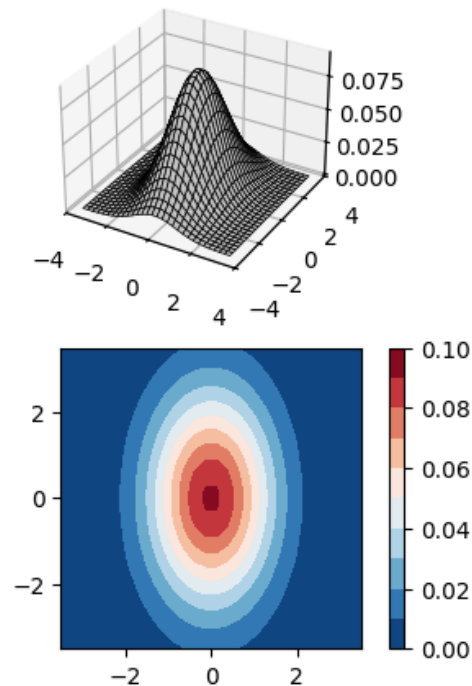
- **Esférica:** $\Sigma = \sigma^2 \mathbf{I}$, con un único parámetro e isodensidades hiperesféricas
- **Diagonal:** $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_D^2)$, D parámetros, isodensidades hiperelipsoidales de semiejes paralelos a la base
- **General:** Σ no diagonal: $D(D+1)/2$ parámetros, isodensidades hiperelipsoidales de semiejes oblicuos a la base

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import multivariate_normal
R = np.linspace(-3.5, 3.5, 30); x, y = np.meshgrid(R, R); me, Se = [0, 0], [[1, 0], [0, 1]]
md, Sd = [0, 0], [[1, 0], [0, 3]]; mg, Sg = [0, 0], [[2., 1.3], [1.3, 2.]]
fig = plt.figure(figsize=(15, 5.25)); fig.tight_layout()
for i, (m, S) in enumerate(zip((me, md, mg), (Se, Sd, Sg)), start=1):
    z = multivariate_normal(m, S).pdf(np.dstack((x, y)))
    ax = fig.add_subplot(2, 3, i, projection='3d'); ax.set_title(f'$S=\{S\}$'.format(S))
    ax.plot_surface(x, y, z, color='white', edgecolor="black", lw=.5)
    ax = fig.add_subplot(2, 3, i+3, aspect='equal')
    cp = ax.contourf(x, y, z, 10, cmap='RdBu_r'); plt.colorbar(cp, ax=ax);
```

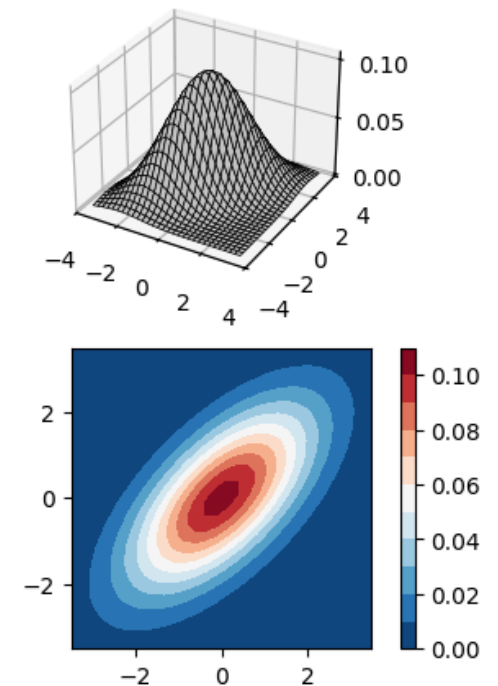
$S = [[1, 0], [0, 1]]$



$S = [[1, 0], [0, 3]]$



$S = [[2.0, 1.3], [1.3, 2.0]]$



11 Entropía

Entropía de una distribución discreta: $\mathbf{y} \in [0, 1]^C, \quad \sum_c y_c = 1$

$$\mathbb{H}(\mathbf{y}) = - \sum_c y_c \log_2 y_c$$

Distribución de máxima entropía: $\mathbf{y} = \frac{1}{C} \mathbf{1}_C$

$$\mathbb{H}(\mathbf{y}) = - \sum_c \frac{1}{C} \log_2 \frac{1}{C} = - \log_2 \frac{1}{C} = \log_2 C$$

Distribución de mínima entropía: $\mathbf{y} = \text{one-hot}(c^*)$ para algún $c^* \in \{1, \dots, C\}$

$$\mathbb{H}(\mathbf{y}) = -1 \log_2 1 - \sum_{c \neq c^*} 0 \log_2 0 = 0$$

Entropía cruzada:

$$\mathbb{H}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_c y_c \log_2 \hat{y}_c \quad \text{cumple} \quad \mathbb{H}(\mathbf{y}, \hat{\mathbf{y}}) \geq \mathbb{H}(\mathbf{y}, \mathbf{y}) = \mathbb{H}(\mathbf{y})$$

Entropía cruzada binaria: entropía cruzada para $C = 2$ clases

$$\mathbb{H}(y, \hat{y}) = -y \log_2 \hat{y} - (1 - y) \log_2 (1 - \hat{y})$$

12 Estimación máximo-verosímil (MLE)

Modelo incondicional: MLE de $\boldsymbol{\theta}$ con respecto a $\mathcal{D} = \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$ iid según $p(\mathbf{y} | \boldsymbol{\theta})$

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{\text{mle}} &= \operatorname{argmax}_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) \quad \text{con} \quad L(\boldsymbol{\theta}) = p(\mathcal{D} | \boldsymbol{\theta}) = \prod_n p(\mathbf{y}_n | \boldsymbol{\theta}) \\ &= \operatorname{argmax}_{\boldsymbol{\theta}} LL(\boldsymbol{\theta}) \quad \text{con} \quad LL(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) = \sum_n \log p(\mathbf{y}_n | \boldsymbol{\theta}) \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} NLL(\boldsymbol{\theta}) \quad \text{con} \quad NLL(\boldsymbol{\theta}) = -LL(\boldsymbol{\theta}) = -\sum_n \log p(\mathbf{y}_n | \boldsymbol{\theta})\end{aligned}$$

- **Bernoulli:** $\mathcal{D} = \{y_1, \dots, y_N\}$, $p(y_n | \theta) = \text{Ber}(y | \theta)$

$$\hat{\theta} = \frac{N_1}{N} \quad \text{con} \quad N_1 = \sum_n \mathbb{I}(y_n = 1)$$

- **Categórica:** $\mathcal{D} = \{y_1, \dots, y_N\}$, $p(y_n | \boldsymbol{\theta}) = \text{Cat}(y | \boldsymbol{\theta})$

$$\hat{\theta}_c = \frac{N_c}{N} \quad \text{con} \quad N_c = \sum_n \mathbb{I}(y_n = c)$$

- **Gaussiana 1d:** $\mathcal{D} = \{y_1, \dots, y_N\}$, $p(y_n | \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2)$

$$\hat{\mu} = \bar{y} \quad \text{y} \quad \hat{\sigma}^2 = \frac{1}{N} \sum_n (y_n - \hat{\mu})^2$$

- **Gaussiana:** $\mathcal{D} = \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$, $p(\mathbf{y}_n | \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y} | \boldsymbol{\mu}, \boldsymbol{\Sigma})$

$$\hat{\boldsymbol{\mu}} = \bar{\mathbf{y}} \quad \text{y} \quad \hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_n (\mathbf{y}_n - \hat{\boldsymbol{\mu}})(\mathbf{y}_n - \hat{\boldsymbol{\mu}})^t$$

Ejercicio: demuestra los casos Bernoulli, categórica y Gaussiana 1d

Bernoulli: $\mathcal{D} = \{y_1, \dots, y_N\}, \quad p(y_n \mid \theta) = \text{Ber}(y \mid \theta)$

$$\begin{aligned}\text{NLL}(\theta) &= -\log \prod_{n=1}^N p(y_n \mid \theta) \\ &= -\log \prod_{n=1}^N \theta^{\mathbb{I}(y_n=1)} (1-\theta)^{\mathbb{I}(y_n=0)} \\ &= -\sum_{n=1}^N \mathbb{I}(y_n = 1) \log \theta + \mathbb{I}(y_n = 0) \log(1-\theta) \\ &= -[N_1 \log \theta + N_0 \log(1-\theta)] \quad \text{con} \quad N_1 = \sum_n \mathbb{I}(y_n = 1) \quad \text{y} \quad N_0 = \sum_n \mathbb{I}(y_n = 0)\end{aligned}$$

$$\frac{d}{d\theta} \text{NLL}(\theta) = \frac{-N_1}{\theta} + \frac{N_0}{1-\theta} \Big|_{\hat{\theta}} = 0 \rightarrow \hat{\theta} = \frac{N_1}{N}$$

Categórica: $\mathcal{D} = \{y_1, \dots, y_N\}, \quad p(y_n \mid \boldsymbol{\theta}) = \text{Cat}(y \mid \boldsymbol{\theta})$

$$\begin{aligned}\text{NLL}(\boldsymbol{\theta}) &= -\log \prod_{n=1}^N p(y_n \mid \boldsymbol{\theta}) \\ &= -\log \prod_{n=1}^N \prod_{c=1}^C \theta_c^{\mathbb{I}(y_n=c)} \\ &= -\sum_{n=1}^N \sum_{c=1}^C \mathbb{I}(y_n = c) \log \theta_c \\ &= -\sum_{c=1}^C N_c \log \theta_c \quad \text{con} \quad N_c = \sum_n \mathbb{I}(y_n = c)\end{aligned}$$

$$\mathcal{L}(\boldsymbol{\theta}, \lambda) = -\sum_c N_c \log \theta_c - \lambda \left(1 - \sum_c \theta_c\right)$$

$$\frac{\partial \mathcal{L}}{\partial \theta_c} = -\frac{N_c}{\theta_c} + \lambda$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -1 + \sum_c \theta_c$$

$$\hat{\theta}_c = \frac{N_c}{N} \quad (\text{igualando a cero las derivadas})$$

Gaussiana 1d: $\mathcal{D} = \{y_1, \dots, y_N\}$, $p(y_n | \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2)$

$$\begin{aligned}\text{NLL}(\mu, \sigma^2) &= -\log \prod_{n=1}^N p(y_n | \theta) \\ &= -\sum_n \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp \left(-\frac{1}{2\sigma^2} (y_n - \mu)^2 \right) \right] \\ &= \frac{N}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_n (y_n - \mu)^2\end{aligned}$$

$$\frac{\partial \text{NLL}}{\partial \mu} = \frac{1}{\sigma^2} \sum_n (\mu - y_n) = \frac{N}{\sigma^2} (\mu - \bar{y}) \Big|_{\hat{\boldsymbol{\theta}}} = 0 \rightarrow \hat{\mu} = \bar{y}$$

$$\frac{\partial \text{NLL}}{\partial \sigma} = \frac{N}{\sigma} - \frac{1}{\sigma^3} \sum_n (y_n - \mu)^2 \Big|_{\hat{\boldsymbol{\theta}}} = 0 \rightarrow \hat{\sigma}^2 = \frac{1}{N} \sum_n (y_n - \hat{\mu})^2$$

Modelo condicional: MLE de θ con respecto a $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$ iid según $p(\mathbf{x}, \mathbf{y} \mid \theta)$

$$\hat{\theta}_{\text{mle}} = \operatorname{argmin}_{\theta} \text{NLL}(\theta) \quad \text{con} \quad \text{NLL}(\theta) = - \sum_n \log p(\mathbf{y}_n \mid \mathbf{x}_n, \theta)$$

- **Cálculo:** suele hacerse iterativamente, mediante descenso por gradiente (estocástico)

Modelo conjunto: MLE de θ con respecto a $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$ iid según $p(\mathbf{x}, \mathbf{y} \mid \theta)$

$$\hat{\theta}_{\text{mle}} = \operatorname{argmin}_{\theta} \text{NLL}(\theta) \quad \text{con} \quad \text{NLL}(\theta) = - \sum_n \log p(\mathbf{x}_n, \mathbf{y}_n \mid \theta)$$

- **Distinto del MLE del modelo condicional:**

$$\begin{aligned} \hat{\theta}_{\text{mle}} &= \operatorname{argmin}_{\theta} - \sum_{n=1}^N \log p(\mathbf{y}_n \mid \mathbf{x}_n, \theta) \\ &= \operatorname{argmin}_{\theta} - \sum_{n=1}^N \log \frac{p(\mathbf{x}_n, \mathbf{y}_n \mid \theta)}{p(\mathbf{x}_n \mid \theta)} \\ &= \operatorname{argmin}_{\theta} - \sum_{n=1}^N \log p(\mathbf{x}_n, \mathbf{y}_n \mid \theta) - \underbrace{\log p(\mathbf{x}_n \mid \theta)}_{\text{depende de toda } \mathbf{y}} \end{aligned}$$

- **Cálculo:** suele ser sencillo, clase a clase, mediante conteo y promediado con los datos de la clase

13 Minimización del riesgo empírico (ERM)

ERM: ERM de θ con respecto a $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$ iid según $p(\mathbf{x}, \mathbf{y} \mid \theta)$, para pérdida genérica $\ell(\mathbf{y}, \hat{\mathbf{y}})$

$$\hat{\theta}_{\text{erm}} = \operatorname{argmin}_{\theta} \mathcal{L}(\theta) \quad \text{con} \quad \mathcal{L}(\theta) = \frac{1}{N} \sum_n \ell(\mathbf{y}_n, \hat{\mathbf{y}}_n)$$

Pérdida 01: $\ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y})$ para clasificadores probabilísticos o no, pero no es diferenciable

Log-pérdida: $\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\log p(\mathbf{y} \mid \mathbf{x}, \theta)$ particulariza la ERM a la MLE (normalizada por N)

$$\mathcal{L}(\theta) = \text{NLL}(\theta) = -\frac{1}{N} \sum_n \log p(\mathbf{y}_n \mid \mathbf{x}_n, \theta)$$

Bernoulli condicional: $y \in \{0, 1\}$, $p(y \mid \mathbf{x}, \theta) = \text{Ber}(y \mid \mu)$, $\hat{y} = \mu = f(\mathbf{x}, \theta)$

$$\ell(y, \hat{y}) = -\log p(y \mid \mathbf{x}, \theta) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}) = \mathbb{H}(y, \hat{y})$$

Categórica condicional: $\mathbf{y} \in \{0, 1\}^C$, $\sum_c y_c = 1$, $p(\mathbf{y} \mid \mathbf{x}, \theta) = \text{Cat}(\mathbf{y} \mid \boldsymbol{\mu})$, $\hat{\mathbf{y}} = \boldsymbol{\mu} = f(\mathbf{x}, \theta)$

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\log p(\mathbf{y} \mid \mathbf{x}, \theta) = -\sum_c y_c \log \hat{y}_c = \mathbb{H}(\mathbf{y}, \hat{\mathbf{y}})$$

Gaussiana (1d) condicional: $y \in \mathbb{R}$, $p(y \mid \mathbf{x}, \theta) = \mathcal{N}(y \mid \mu, \sigma^2)$, $\hat{y} = \mu = f(\mathbf{x}, \theta)$, σ^2 fija

$$\ell(y, \hat{y}) = -\log p(y \mid \mathbf{x}, \theta) = -\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y - \hat{y})^2 \equiv (y - \hat{y})^2$$

Ejercicio: halla la derivada (o gradiente) de la log-pérdida respecto de la predicción en los tres casos anteriores

Entropía cruzada binaria (Bernoulli condicional): $\ell(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

$$\begin{aligned}\frac{\partial \ell(y, \hat{y})}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}\end{aligned}$$

Entropía cruzada (categórica condicional): $\ell(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_c y_c \log \hat{y}_c = -\mathbf{y}^t \log(\hat{\mathbf{y}})$

$$\begin{aligned}\frac{\partial \ell(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} &= \left(\frac{\partial \ell(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{y}_1}, \dots, \frac{\partial \ell(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{y}_C} \right) \\ &= \left(-\frac{y_1}{\hat{y}_1}, \dots, -\frac{y_C}{\hat{y}_C} \right) \\ &= -\mathbf{y}^t \oslash \hat{\mathbf{y}}^t \quad (\text{división Hadamard})\end{aligned}$$

Pérdida cuadrática (Gaussiana (1d) condicional): $\ell_2(y, \hat{y}) = (y - \hat{y})^2$

$$\frac{\partial \ell_2(y, \hat{y})}{\partial \hat{y}} = 2(\hat{y} - y)$$

14 Regularización

Sobre-entrenamiento (overfitting): problema fundamental de MLE y, en general, ERM, asociado a la minimización de la pérdida sobre los datos de entrenamiento, ya que conduce a modelos **sobre-entrenados** que **no generalizan** (bien)

Regularización: añade una **penalización (penalty)** a la pérdida mediante alguna forma de **penalización de complejidad** $C(\boldsymbol{\theta})$ cuyo peso depende de un **parámetro de regularización** $\lambda \geq 0$

$$\mathcal{L}(\boldsymbol{\theta}, \lambda) = \frac{1}{N} \sum_n \ell(\mathbf{y}_n, \hat{\mathbf{y}}_n) + \lambda C(\boldsymbol{\theta})$$

Penalización de complejidad usual: asume dado un **prior** más o menos plano, $p(\boldsymbol{\theta})$

$$C(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$$

Estimación maximum a posteriori (MAP): asumiendo $p(\boldsymbol{\theta} \mid \mathcal{D}) = \delta(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_{\text{map}})$ y log-pérdida

$$\hat{\boldsymbol{\theta}}_{\text{map}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log p(\boldsymbol{\theta} \mid \mathcal{D}) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \text{NLL}(\boldsymbol{\theta}) - \lambda \log p(\boldsymbol{\theta})$$

- λ **demasiado pequeño:** equivale a minimizar el riesgo empírico y acabar con un modelo **sobre-ajustado**
- λ **demasiado grande:** equivale a no desviarse mucho del prior y acabar con un modelo **sub-ajustado**

Dificultad para estimar λ : con los datos de entrenamiento obtenemos $\hat{\lambda} = 0$

Validación: método más popular y sencillo para escoger λ

- Dividimos los datos en un conjunto de **entrenamiento** $\mathcal{D}_{\text{train}}$ y otro de **validación o desarrollo** $\mathcal{D}_{\text{valid}}$
- Cada valor de λ se evalúa con $\mathcal{D}_{\text{valid}}$ tras ajustar el modelo con $\mathcal{D}_{\text{train}}$
- Elegimos el λ que proporciona mejor rendimiento en $\mathcal{D}_{\text{valid}}$

K -fold cross validation (K -CV): versión refinada de validación que aprovecha todos los datos para validar

- Dividimos los datos en K bloques, $\mathcal{D}_1, \dots, \mathcal{D}_K$
- Cada valor de λ se evalúa con cada bloque $\mathcal{D}_k$ tras ajustar el modelo con el resto de bloques, $\mathcal{D}_{-k}$
- Elegimos el λ que proporciona mejor rendimiento promedio

Leave-one-out (LOO): K -CV con tantos bloques como datos

Estimación de hiperparámetros en general: también puede usarse validación, K -CV y LOO

Otras técnicas usuales de regularización:

- **Terminación temprana (early stopping):** si se usa un algoritmo iterativo de entrenamiento, podemos validar el rendimiento del modelo tras cada iteración y terminar antes de convergencia si se observa sobre-entrenamiento
- **Uso de más datos:** dado un modelo de complejidad fija, la posibilidad de sobre-ajuste decrece al aumentar los datos

Estimación del riesgo teórico: mediante un conjunto de **test** $\mathcal{D}_{\text{test}}$ separado de los datos

15 Decisión

Inferencia Bayesiana: cálculo de la **posterior** $p(H \mid \mathbf{x})$ mediante la regla de Bayes actualizar nuestras creencias sobre cantidades ocultas H a partir de datos $\mathbf{x}$

Teoría de la decisión Bayesiana: usa la inferencia para decidir cuál es la mejor de las posibles **acciones** a realizar

Agente: debe escoger una acción de un conjunto de acciones posibles, $\mathcal{A}$

Estado de la naturaleza: $h \in \mathcal{H}$, condiciona los costes y beneficios que se derivan de tomar cada acción posible

Función de pérdida: indica el coste incurrido al tomar la acción $a \in \mathcal{A}$ cuando el estado de la naturaleza es $h \in \mathcal{H}$

$$\ell(h, a)$$

Riesgo (pérdida) esperado a posteriori: de a tras observar $\mathbf{x}$

$$R(a \mid \mathbf{x}) = \mathbb{E}_{p(h|\mathbf{x})}[\ell(h, a)] = \sum_{h \in \mathcal{H}} \ell(h, a) p(h \mid \mathbf{x})$$

Política óptima o estimador de Bayes: obtiene una acción de mínimo riesgo por cada observación posible

$$\pi^*(\mathbf{x}) = \operatorname{argmin}_{a \in \mathcal{A}} R(a \mid \mathbf{x})$$

Problemas de clasificación

Estados de la naturaleza y acciones: etiquetas de clase, $\mathcal{H} = \mathcal{Y} = \{1, \dots, C\}$ y $\mathcal{A} = \mathcal{Y}$

Pérdida 01 esperada a posteriori: la probabilidad de error a posteriori es uno menos la de acertar a posteriori

$$R(\hat{y} \mid \mathbf{x}) = \sum_y \ell_{01}(y, \hat{y}) p(y \mid \mathbf{x}) = \sum_{y \neq \hat{y}} p(y \mid \mathbf{x}) = 1 - p(\hat{y} \mid \mathbf{x})$$

Estimador de Bayes: $\pi(\mathbf{x}) = \operatorname{argmax}_{y \in \mathcal{Y}} p(y \mid \mathbf{x})$ **estimador MAP o moda** de la posterior

Matriz de confusión: $\mathcal{D} = \{(y_m, \hat{y}_m)\}$, $\mathbf{M} = [M_{y,\hat{y}}]$ con $M_{y,\hat{y}} = \sum_m \mathbb{I}(y_m = y) \mathbb{I}(\hat{y}_m = \hat{y})$

y	$\hat{1}$	$\hat{2}$	$\dots$	$\hat{C}$	Suma fila
1	$M_{1,\hat{1}}$	$M_{1,\hat{2}}$	$\dots$	$M_{1,\hat{C}}$	$M_{1,:}$
2	$M_{2,\hat{1}}$	$M_{2,\hat{2}}$	$\dots$	$M_{2,\hat{C}}$	$M_{2,:}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
C	$M_{C,\hat{1}}$	$M_{C,\hat{2}}$	$\dots$	$M_{C,\hat{C}}$	$M_{C,:}$
Suma:	$M_{:, \hat{1}}$	$M_{:, \hat{2}}$	$\dots$	$M_{:, \hat{C}}$	M

Normalización por filas: estimación empírica de $p(\hat{y} \mid y)$

Normalización por columnas: estimación empírica de $p(y \mid \hat{y})$

Normalización por filas y columnas: estimación empírica de $p(y, \hat{y})$

Análisis de una clase específica: se reduce a matriz binaria tomando el resto de clases como clase negativa

Problemas de regresión

Estados de la naturaleza y acciones: reales, $\mathcal{H} = \mathcal{A} = \mathcal{Y} = \mathbb{R}$

Pérdida L2 (ℓ_2 , cuadrática o error cuadrático): $\ell_2(h, a) = (h - a)^2$

Pérdida L2 esperada a posteriori:

$$R(a \mid \mathbf{x}) = \mathbb{E}[(h - a)^2 \mid \mathbf{x}] = \mathbb{E}[h^2 \mid \mathbf{x}] - 2a\mathbb{E}[h \mid \mathbf{x}] + a^2$$

Regresor de Bayes L2 o minimum mean squared error (MMSE): media a posteriori

$$\frac{\partial}{\partial a} R(a \mid \mathbf{x}) = -2\mathbb{E}[h \mid \mathbf{x}] + 2a = 0 \quad \rightarrow \quad \pi(\mathbf{x}) = \mathbb{E}[h \mid \mathbf{x}] = \int h p(h \mid \mathbf{x}) dh$$